

W13D04 — SELECT, JOIN & Query Patterns

ML Engineer Journey | Phase 4 — Database & Observability | 4 Jun 2026

1. DQL dan Urutan Eksekusi SQL

DQL (Data Query Language) adalah subset SQL yang tugasnya satu: membaca data tanpa mengubah apapun di database. Klausa utamanya adalah SELECT, FROM, JOIN, WHERE, GROUP BY, HAVING, ORDER BY, dan LIMIT. Yang kritis untuk dipahami: urutan **penulisan** tidak sama dengan urutan **eksekusi** PostgreSQL.

Urutan eksekusi PostgreSQL:

Urutan	Klausa	Yang Dilakukan	Catatan Penting
①	FROM / JOIN	Ambil tabel sumber, bentuk dataset awal	Relasi foreign key dipakai di sini
②	WHERE	Filter baris individual	Tidak bisa pakai alias dari SELECT, tidak bisa pakai agregat
③	GROUP BY	Kelompokkan baris	Semua non-agregat di SELECT harus ada di sini
④	HAVING	Filter grup hasil GROUP BY	Bisa pakai COUNT, SUM, AVG — inilah bedanya dengan WHERE
⑤	SELECT	Pilih kolom output, buat alias	Alias baru tersedia setelah tahap ini
⑥	ORDER BY	Urutkan hasil	Satu-satunya klausa yang bisa pakai alias dari SELECT
⑦	LIMIT / OFFSET	Potong jumlah baris output	Dieksekusi paling akhir

Konsekuensi langsung: *WHERE* tidak bisa memfilter berdasarkan hasil *COUNT* atau *SUM* — karena agregasi belum terjadi saat *WHERE* dijalankan. Gunakan *HAVING* untuk kondisi berbasis fungsi agregat.

2. SELECT Dasar — Eksplisit Kolom

Aturan pertama SELECT di production: jangan pernah pakai **SELECT ***. Mengambil semua kolom berarti membebani network dengan data yang tidak dibutuhkan, memperlambat deserialization di sisi Python, dan membuat kode rapuh saat schema berubah.

```
-- Benar: eksplisit kolom
SELECT id, model_name, model_version, accuracy, created_at
FROM models
WHERE is_active = TRUE
ORDER BY accuracy DESC
LIMIT 10;
```

LIMIT wajib ada saat query ke tabel besar. Tanpa LIMIT, seluruh tabel (bisa jutaan baris) dikirim ke Python sekaligus — ini recipe untuk OOM error di production.

3. JOIN — Menggabungkan Data Antar Tabel

JOIN memanfaatkan relasi foreign key yang kamu definisikan saat DDL (W13D02). Ada empat jenis JOIN, tapi 90% kebutuhan production hanya butuh dua: INNER JOIN dan LEFT JOIN.

Tipe JOIN	Yang Dikembalikan	Kapan Dipakai	NULL di output?
-----------	-------------------	---------------	-----------------

INNER JOIN	Hanya baris yang cocok di kedua tabel	EMBA data lengkap dari kedua tabel	Tidak
LEFT JOIN	Semua baris tabel kiri + matchManusia semua induk, meski tanpa anak	Ya	kolom kanan NULL
RIGHT JOIN	Semua baris tabel kanan + matchMakhluk — tulis ulang jadi LEFT JOIN	Ya	kolom kiri NULL
FULL JOIN	Semua baris dari kedua tabel	Rekonsiliasi data, audit mismatch	match — kedua sisi bisa NULL

Contoh INNER JOIN dengan alias tabel:

```
SELECT
  m.model_name,
  m.model_version,
  p.prediction_result,
  p.confidence_score
FROM models m          -- alias 'm'
INNER JOIN predictions p -- alias 'p'
  ON p.model_id = m.id  -- kondisi join: foreign key
WHERE p.confidence_score > 0.8
ORDER BY p.created_at DESC;
```

Contoh LEFT JOIN + GROUP BY — semua model termasuk yang belum punya prediksi:

```
SELECT
  m.model_name,
  COUNT(p.id)          AS total_predictions, -- COUNT(kolom), bukan COUNT(*)
  AVG(p.confidence_score) AS avg_confidence
FROM models m
LEFT JOIN predictions p ON p.model_id = m.id
GROUP BY m.id, m.model_name, m.model_version
ORDER BY total_predictions DESC;
```

COUNT(p.id) bukan COUNT() — ini penting. COUNT(*) menghitung semua baris termasuk NULL yang datang dari LEFT JOIN. COUNT(p.id) hanya menghitung baris yang p.id-nya tidak NULL, sehingga model tanpa prediksi benar-benar mengembalikan 0, bukan angka palsu.*

4. GROUP BY dan HAVING

GROUP BY mengubah cara kerja SELECT secara fundamental: dari 'tampilkan setiap baris' menjadi 'tampilkan satu baris per grup'. Setiap kolom di SELECT yang bukan fungsi agregat wajib ada di GROUP BY — PostgreSQL akan error jika tidak.

```
SELECT
  m.model_name,
  DATE(p.created_at)    AS prediction_date,
  COUNT(p.id)           AS total,
  AVG(p.confidence_score) AS avg_confidence
FROM predictions p
INNER JOIN models m ON m.id = p.model_id
```

```

GROUP BY
    m.model_name,
    DATE(p.created_at)    -- harus sesuai dengan non-agregat di SELECT
HAVING COUNT(p.id) > 5    -- filter SETELAH agregasi
ORDER BY prediction_date DESC;

```

	WHERE	HAVING
Filter pada	Baris individual	Grup (hasil GROUP BY)
Dieksekusi	Sebelum GROUP BY	Setelah GROUP BY
Bisa fungsi agregat?	Tidak	Ya (COUNT, SUM, AVG...)
Contoh valid	WHERE status = 'active'	HAVING COUNT(*) > 10
Performance	Lebih cepat — filter awal	Lebih lambat — filter akhir

Jika kondisi bisa ditulis di WHERE, tulis di WHERE. HAVING hanya untuk kondisi yang membutuhkan nilai hasil fungsi agregat.

5. EXPLAIN ANALYZE — Membaca Query Plan

EXPLAIN menunjukkan rencana eksekusi PostgreSQL tanpa benar-benar menjalankan query. EXPLAIN ANALYZE menjalankan query sungguhan dan menampilkan timing aktual. Gunakan EXPLAIN ANALYZE untuk debugging performa, tapi hati-hati di production karena query benar-benar dieksekusi.

```

EXPLAIN ANALYZE
SELECT m.name, p.prediction, p.confidence
FROM predictions p
INNER JOIN models m ON p.model_id = m.id
WHERE p.confidence > 0.8;

```

Cara membaca output EXPLAIN:

Istilah	Artinya	Implikasi
Seq Scan	Baca seluruh tabel baris per baris	Tidak ada index yang dipakai — $O(n)$
Index Scan	Lompat ke baris target via index	Index dipakai — $O(\log n)$
cost=X..Y	Estimasi cost (bukan detik)	Y adalah total cost — lebih kecil lebih baik
rows=N	Estimasi jumlah baris diproses	Sangat meleset jika ANALYZE belum dijalankan
actual rows=N	Jumlah baris aktual (hanya di ANALYZE)	Bandingkan dengan estimasi untuk cek akurasi planner
Hash Join	Bangun hash table dari satu sisi, probe dari sisi lain	Effisien jika sisi kanan ter-index
Nested Loop	Untuk setiap baris kiri, cari baris kanan	Effisien jika sisi kanan kecil atau ter-index

6. Index — Kapan Dibuat, Kapan Tidak Efektif

Index bekerja seperti indeks buku: daripada baca setiap halaman, kamu langsung lompat ke nomor halaman yang dicari. Secara teknis, PostgreSQL menggunakan B-tree (balanced tree) — lookup $O(\log$

n) versus $O(n)$ untuk sequential scan.

```
-- Buat index pada kolom yang sering di-filter atau di-JOIN
CREATE INDEX IF NOT EXISTS idx_predictions_model_id
    ON predictions(model_id);

CREATE INDEX IF NOT EXISTS idx_predictions_confidence
    ON predictions(confidence);

-- Update statistik tabel agar planner lebih akurat
ANALYZE predictions;
```

Kondisi	PostgreSQL pilih Index atau Sequential Scan	Alasan
Tabel sangat kecil (<1000 baris)	Tidak — tetap Seq Scan	Overhead B-tree lookup > keuntungan
Filter lolos <15% baris (high selectivity)	Tidak — tetap Seq Scan	Index efektif — sedikit baris yang match
Filter lolos >30% baris (low selectivity)	Tidak — tetap Seq Scan	Seq Scan lebih cepat — mudah baca linear daripada hop-hop
Kolom foreign key (JOIN condition)	Tidak — tetap Seq Scan	Index sangat membantu JOIN performance
Kolom ORDER BY tanpa filter	Kadang	Tergantung ukuran tabel dan cost estimasi

Jangan buat index di semua kolom. Index menambah overhead saat INSERT/UPDATE/DELETE karena B-tree harus di-update. Prioritas: kolom yang sering jadi kondisi JOIN, kolom yang sering di-filter WHERE, dan kolom timestamp yang sering di-ORDER BY.

7. Pola Psycopg2 untuk Query

Semua pattern berikut sudah divalidasi dari sesi ini. Gunakan sebagai referensi saat menulis fungsi query baru.

```
# Pola standar: with cursor, parameterized query, list comprehension
def get_data(conn, threshold: float) -> list[dict]:
    sql = """
        SELECT m.model_name, COUNT(p.id) AS total
        FROM models m
        LEFT JOIN predictions p ON p.model_id = m.id
        WHERE m.is_active = TRUE
        GROUP BY m.id, m.model_name
        HAVING COUNT(p.id) > %s
        ORDER BY total DESC;
    """

    with conn.cursor() as cur:          # auto-close cursor
        cur.execute(sql, (threshold,)) # parameterized, aman dari injection
        rows = cur.fetchall()

    return [
        {"name": r[0], "total": r[1]}
        for r in rows
    ]

# fetchone() – cek not-found
```

```
cur.execute("SELECT * FROM models WHERE id = %s", (model_id,))
row = cur.fetchone()
if row is None:
    raise ValueError(f"Model {model_id} tidak ditemukan")
```

Ringkasan — Yang Paling Penting

1. Urutan eksekusi SQL: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. Bukan urutan penulisan.
2. SELECT * berbahaya di production. Selalu eksplisit kolom.
3. LIMIT wajib untuk tabel besar — tanpa LIMIT semua baris dikirim ke Python.
4. COUNT(kolom) vs COUNT(*): COUNT(kolom) mengabaikan NULL, COUNT(*) tidak. Dalam LEFT JOIN, keduanya menghasilkan angka berbeda.
5. WHERE untuk filter baris, HAVING untuk filter grup setelah agregasi. WHERE tidak bisa pakai hasil COUNT/SUM.
6. Semua kolom non-agregat di SELECT wajib ada di GROUP BY — PostgreSQL error jika tidak.
7. EXPLAIN ANALYZE benar-benar menjalankan query. Gunakan EXPLAIN saja untuk check plan tanpa eksekusi.
8. Seq Scan pada tabel kecil adalah normal dan benar. Index baru worth it mulai ratusan ribu baris.
9. Setelah INSERT/UPDATE besar, jalankan ANALYZE table_name agar planner memiliki statistik akurat.
10. JOIN tanpa kondisi ON menghasilkan Cartesian product — semua baris kali semua baris. Selalu pastikan ada ON.